

Dimensionality Reduction Approaches to Insider Threat Detection: From Principal Components to Interpretable Sparse Features

by

Suramya Raj Angdembay

A Thesis

Submitted to the Honors College of
The University of Southern Mississippi
in Partial Fulfillment
of Honors Requirements

Month Year

Approved by:

Dr. Haiyan Tian, Degree

School of Mathematics and Natural Sciences

Thesis Co-Advisor Name, Degree

School of Your School

Director Name, Degree

School of Your School

Joyce Inman, Ph.D., Dean

Honors College

ABSTRACT

Insider threat represents a major cybersecurity challenge to companies, organizations, and government agencies. Detecting malicious insiders is difficult due to massive data volumes, extreme class imbalance, and the subtle nature of behavioral shifts over time. This thesis approaches the problem through dimensionality reduction: describing high-dimensional behavior with a small number of informative directions, and treating what a model of normal behavior fails to explain as a sign of anomaly. First, a classical approach uses Deep Feature Synthesis (DFS) to generate features, Principal Component Analysis (PCA) to reduce them to their dominant directions of variance, and a Gaussian Mixture Model (GMM) to score observations that fall outside that structure. Second, a large language model is fine-tuned only on benign user-days written as text, with the fine-tuning restricted to low-rank changes of the model’s weights, and it scores each day by its average surprisal, the negative log-probability of each token given the tokens before it. We show that this score splits exactly into a profile part, covering organizational and personality attributes, and a behavior part, and that the profile part ranks users backwards: on CERT release 4.2, profile tokens alone place malicious users below benign ones (user-level AUC 0.47), while scoring behavior tokens alone raises user-level AUC from 0.65 to 0.86. Third, we apply an interpretable form of dimensionality reduction to the model itself. A sparse autoencoder, an overcomplete and sparse relative of PCA, is learned on the change that fine-tuning made to the model’s internal representations. Four of the five features most associated with malicious days activate almost only on behavioral tokens. On malicious users held out from feature selection, moving these features toward a benign colleague’s values rather than another malicious user’s lowers the anomaly

score more than the same edit of control features does, by a small margin. Where a feature activates, what it represents and which predictions its edit changes are separate questions; the last is the subject of an experiment still in progress. All experiments use the CMU CERT insider threat dataset.

Keywords: Insider Threat; Dimensionality Reduction; Principal Component Analysis (PCA); Low-Rank Adaptation; Surprisal; Large Language Models; Mechanistic Interpretability; Sparse Autoencoders; One-Class Anomaly Detection

DEDICATION

Your dedication text here.

ACKNOWLEDGMENTS

Your acknowledgments begin here. Recognize advisors, faculty, funding agencies, and institutional support.

Contents

ABSTRACT	2
DEDICATION	4
ACKNOWLEDGMENTS	5
LIST OF ABBREVIATIONS	13
1 Introduction	14
1.1 Example 1st Level Heading	14
Example 2nd Level Heading	14
2 Principal Component Analysis	15
2.1 Overview and Motivation	15
2.2 PCA in the p -Dimensional Setting	15
Data assumptions for PCA	15
Data model and centering	16
Covariance matrix	18
Variance maximization and eigenvalue problem	18
Eigendecomposition and ordering	19
Dimensionality reduction by projection	20
Explained variance	20
2.3 Reconstruction and Reconstruction Error	21
Orthogonal reconstruction	21

	Reconstruction error as distance to the principal subspace	21
2.4	A Step-by-Step Numerical Example of PCA and Reconstruction	22
	Step 1: Compute the Sample Mean	22
	Step 2: Center the Data	23
	Step 3: Compute the Covariance Matrix	23
	Step 4: Compute Eigenvalues and Eigenvectors	23
	Step 5: Project the Data onto the First Principal Component	24
	Step 6: Reconstruct the Data from the Reduced Representation	24
	Step 7: Compute Reconstruction Error	25
	Interpretation	25
2.5	Low-Dimensional Geometric Intuition	26
	A two-dimensional illustration	26
	A three-dimensional illustration	26
2.6	A High-Dimensional Example: A 17-Dimensional Dataset	27
2.7	Synthetic anomaly example	28
	Original Behavioral Space	29
	Projection into PCA Space	30
	Principal Component Coordinates	31
	Summary of the Illustration	33
2.8	Summary	34
3	Experimental Setup and Datasets	35
3.1	The CMU CERT Dataset	35
3.2	Data Preparation for Static PCA Analysis	35
3.3	Data Preparation for Language-Model Analysis	35
	Serialization	35
	Model and Training	36
	User-Disjoint Evaluation	36

4	Language-Model Surprisal and Its Decomposition	37
4.1	Overview and Motivation	37
4.2	Writing a User-Day as Text	37
	What the class of a token means	38
	Which class a surprisal is counted in	39
4.3	Next-Token Probability and Surprisal	40
	Step 1: Summarize the context as a vector	40
	Step 2: Score every candidate token	41
	Step 3: Turn scores into probabilities with the softmax	42
	Step 4: From probability to surprisal	43
4.4	Why the Negative Logarithm Measures Surprise	44
	Two requirements	45
	The requirements force the logarithm	45
	What the formula says	46
	A worked example: coin flips	46
	Surprisal and the negative log-likelihood	47
	The surprisal of a whole day is its negative log-likelihood	47
4.5	Learning From Benign Days Only	48
	What minimizing surprisal learns	48
	Low-rank adaptation	49
4.6	The Anomaly Score of a Day and of a User	50
4.7	Splitting the Score by Token Class	51
	The exact decomposition	51
	The behavior-only score	51
	Why subtracting the profile mean does not work	52
4.8	A Step-by-Step Numerical Example	52
	Step 1: Token Probabilities and Surprisals	53

Step 2: Class Totals	53
Step 3: Full Scores	53
Step 4: Class Means and the Decomposition Check	53
Step 5: Where the Full Score Went Wrong	54
Step 6: The Naive Subtraction Reverses a Ranking	54
4.9 Measuring Detection: The User-Level AUC	55
4.10 Core Finding	55
5 Interpretable Dimensionality Reduction of the Fine-Tuned Model	58
5.1 Overview and Motivation	58
5.2 Hidden States and the Adaptation Difference	58
5.3 A Sparse Dictionary for the Difference	59
5.4 A Step-by-Step Numerical Example of Sparse Coding	60
Step 1: Encode	61
Step 2: Keep the Top $k = 1$	61
Step 3: Decode and Measure the Error	61
Step 4: Compare With the Two Axes Alone	61
Step 5: A Different Token Uses a Different Feature	61
5.5 Choosing the Features to Test	61
5.6 Where Features Fire: Token-Class Attribution	62
5.7 Causal Test: Editing Features and Re-Scoring	63
Step 1: Encode the receiver	63
Step 2: Move the selected features toward the donor	63
Step 3: Decode and replace the hidden state	63
Step 4: Re-score	64
Step 5: Keep the best candidate	64
Step 6: Compare donors, then compare feature sets	64
5.8 Core Mechanistic Finding	65

Where the selected features activate	65
Editing them changes the score, on held-out users	66
Reading	67
5.9 Limitations	68
REFERENCES	69

List of Tables

4.1	Scored tokens by line for the record above. The record has 213 tokens; the first token, DAY , is never scored because nothing comes before it.	39
4.2	User-level AUC on CERT release 4.2 (60 malicious users, 79 benign users, none seen in training) for scores built from different token classes. Intervals are 95% bootstrap intervals, resampling malicious users.	56
5.1	PCA and the sparse autoencoder side by side.	60
5.2	Share of each selected feature’s activation on each token class, over malicious days of CERT release 4.2 (layer 26). Session (SES) lines are 74.9% of these tokens, the organization (DAY) line 12.6%, and the personality (PSY) line 9.0%.	66

List of Figures

2.1	Illustration of PCA in two dimensions: PCA rotates the coordinate system so that PC1 captures the largest variance while PC2 captures the smallest. Projecting onto PC1 yields a one-dimensional representation with minimal loss when PC2 contributes little variance.	27
2.2	Illustration of PCA in three dimensions: PC1, PC2, and PC3 form an orthonormal basis with decreasing explained variance. Dropping PC3 projects data onto a best-fit plane.	28
2.3	A high-dimensional PCA illustration (17 features): PCA reveals dominant axes of variation among observations (countries). Large separation along PC1 indicates an observation that differs substantially along the primary pattern in the data.	29
2.4	Synthetic two-dimensional dataset with principal component directions drawn from the center of the data.	30
2.5	Dataset represented in the PCA coordinate system where each observation is described by its PC1 and PC2 coordinates.	31
2.6	Values of the first principal component for all observations.	32
2.7	Values of the second principal component for all observations.	33

LIST OF ABBREVIATIONS

APA	American Psychological Association
AUC	Area Under the (Receiver Operating Characteristic) Curve
CERT	CERT Division of the Software Engineering Institute
DFS	Deep Feature Synthesis
GMM	Gaussian Mixture Model
LoRA	Low-Rank Adaptation
PCA	Principal Component Analysis
QLoRA	Quantized Low-Rank Adaptation
SAE	Sparse Autoencoder
USM	The University of Southern Mississippi

Chapter I

Introduction

Begin the first chapter here. Indent the first line of each paragraph.

1.1 Example 1st Level Heading

Text for first-level heading.

Example 2nd Level Heading

Text for second-level heading.

Example 3rd Level Heading (In Paragraph)

Example 3rd Level Heading. Continue paragraph text immediately after the bold heading.

Chapter II

Principal Component Analysis

2.1 Overview and Motivation

Principal Component Analysis (PCA) is a classical technique for identifying orthogonal directions along which the data exhibits the greatest variance. In applications such as behavioral cybersecurity logs, the raw feature space can be high-dimensional and strongly correlated, which complicates visualization, statistical modeling, and anomaly detection. PCA addresses these issues by constructing an orthonormal coordinate system whose axes (the *principal components*) are ordered by the amount of variance they explain. This chapter presents the mathematical formulation of PCA in the general p -dimensional setting, followed by a sequence of low- to high-dimensional illustrations to develop geometric intuition. We conclude with a reconstruction-error viewpoint that motivates PCA-based anomaly scoring. Historically, the geometric origins of PCA trace back to Pearson, while the familiar algebraic eigenvalue formulation was developed by Hotelling [1, 2, 3].

2.2 PCA in the p -Dimensional Setting

Data assumptions for PCA

PCA operates under a set of assumptions about the structure of the data, independent of the specific preprocessing steps used:

- **Vector-space representation:** Each observation must be representable as a numerical vector in $\mathbb{R}^p$, so that linear algebraic operations such as inner products and projections are well-defined.
- **Linearity of structure:** PCA assumes that the relationships between features can be captured through linear combinations, and that the data can be meaningfully approximated by a linear subspace.
- **Variance as informative structure:** PCA assumes that directions with larger variance correspond to meaningful variation in the data, and therefore prioritizes these directions in constructing the principal components.
- **Euclidean geometry:** PCA relies on Euclidean distance, both in defining variance and in measuring reconstruction error. This assumes that distances between observations are meaningful in the feature space.
- **Comparable feature scales:** PCA is sensitive to the relative magnitude of features, and therefore needs standardization of the features.

Data model and centering

Let $X \in \mathbb{R}^{n \times p}$ be a data matrix with n observations and p features,

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}.$$

The i th observation is the row vector $\mathbf{x}_i^\top \in \mathbb{R}^{1 \times p}$. Equivalently, $\mathbf{x}_i \in \mathbb{R}^p$ denotes the corresponding column-vector form of the same observation.

The sample mean vector is defined as

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = \begin{bmatrix} \bar{x}_1 \\ \bar{x}_2 \\ \vdots \\ \bar{x}_p \end{bmatrix} \in \mathbb{R}^p,$$

where each scalar component $\bar{x}_j$ is the mean of the j th feature.

Define the vector of ones

$$\mathbf{1} = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} \in \mathbb{R}^n.$$

The outer product $\mathbf{1}\bar{\mathbf{x}}^\top$ constructs an $n \times p$ matrix whose rows all equal the mean vector:

$$\mathbf{1}\bar{\mathbf{x}}^\top = \begin{bmatrix} \bar{x}_1 & \bar{x}_2 & \cdots & \bar{x}_p \\ \bar{x}_1 & \bar{x}_2 & \cdots & \bar{x}_p \\ \vdots & \vdots & \ddots & \vdots \\ \bar{x}_1 & \bar{x}_2 & \cdots & \bar{x}_p \end{bmatrix}.$$

The centered data matrix is therefore defined as

$$X_c = X - \mathbf{1}\bar{\mathbf{x}}^\top,$$

which subtracts the sample mean vector from each observation. In component form,

$$X_c = \begin{bmatrix} x_{11} - \bar{x}_1 & x_{12} - \bar{x}_2 & \cdots & x_{1p} - \bar{x}_p \\ x_{21} - \bar{x}_1 & x_{22} - \bar{x}_2 & \cdots & x_{2p} - \bar{x}_p \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} - \bar{x}_1 & x_{n2} - \bar{x}_2 & \cdots & x_{np} - \bar{x}_p \end{bmatrix}.$$

Covariance matrix

The sample covariance matrix is

$$S = \frac{1}{n-1} X_c^\top X_c \in \mathbb{R}^{p \times p}. \quad (2.1)$$

The matrix S is symmetric and positive semidefinite, hence it admits an orthonormal eigen-decomposition.

Variance maximization and eigenvalue problem

The covariance matrix summarizes how variation is distributed across the feature space, so the next natural question is which direction captures as much of that variation as possible. PCA answers this by projecting the centered observations onto a unit vector and choosing the direction for which the projected scores have maximum variance. This variance-maximization view leads directly to the eigenvalue problem for S [2, 3].

Consider a unit direction vector $\mathbf{a} \in \mathbb{R}^p$ with $\mathbf{a}^\top \mathbf{a} = 1$. Projecting each centered observation onto $\mathbf{a}$ yields scalar scores

$$z_i = \mathbf{a}^\top (\mathbf{x}_i - \bar{\mathbf{x}}), \quad i = 1, \dots, n.$$

Because the data is centered, the mean of these scores is zero. The empirical variance of

these projected scores is

$$\text{Var}(z) = \frac{1}{n-1} \sum_{i=1}^n z_i^2 = \mathbf{a}^\top \left(\frac{1}{n-1} X_c^\top X_c \right) \mathbf{a} = \mathbf{a}^\top S \mathbf{a}.$$

PCA selects the first principal direction by solving

$$\max_{\mathbf{a} \in \mathbb{R}^p} \mathbf{a}^\top S \mathbf{a} \quad \text{subject to} \quad \mathbf{a}^\top \mathbf{a} = 1. \quad (2.2)$$

Using a Lagrange multiplier λ ,

$$\mathcal{L}(\mathbf{a}, \lambda) = \mathbf{a}^\top S \mathbf{a} - \lambda(\mathbf{a}^\top \mathbf{a} - 1),$$

and differentiating with respect to $\mathbf{a}$ gives

$$S \mathbf{a} = \lambda \mathbf{a}. \quad (2.3)$$

Thus, principal directions are eigenvectors of S , and the corresponding eigenvalues equal the variance explained along those directions. We denote these principal directions by $\mathbf{u}_1, \dots, \mathbf{u}_p$ in the eigendecomposition below.

Eigendecomposition and ordering

Let the eigenvalues be ordered as

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0,$$

with associated orthonormal eigenvectors $\mathbf{u}_1, \dots, \mathbf{u}_p$. Collect them into

$$U = [\mathbf{u}_1 \ \mathbf{u}_2 \ \dots \ \mathbf{u}_p] \in \mathbb{R}^{p \times p}, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_p).$$

Then

$$S = U\Lambda U^\top.$$

Dimensionality reduction by projection

For a target dimension $q < p$, we restrict our new coordinate system to the first q principal directions (the eigenvectors corresponding to the largest eigenvalues). We collect these into a projection matrix:

$$U_q = [\mathbf{u}_1 \ \mathbf{u}_2 \ \cdots \ \mathbf{u}_q] \in \mathbb{R}^{p \times q}.$$

To reduce the dimensionality of the i th centered observation $\mathbf{x}_{c,i} \in \mathbb{R}^p$, we project it onto this q -dimensional subspace. The new coordinates, or *principal component scores*, are given by the inner products of the observation with each of the q eigenvectors:

$$\mathbf{z}_i = U_q^\top \mathbf{x}_{c,i} \in \mathbb{R}^q.$$

Applying this transformation simultaneously to all n observations in our centered data matrix X_c yields the reduced representation (score matrix) Z :

$$Z = X_c U_q \in \mathbb{R}^{n \times q}. \tag{2.4}$$

Here, the i th row of Z is precisely the transposed score vector $\mathbf{z}_i^\top$.

Explained variance

The total variance in the dataset is equivalent to the trace of the covariance matrix S , which is equal to the sum of all its eigenvalues, $\sum_{j=1}^p \lambda_j$. Because the variance captured by the j th principal component is exactly its corresponding eigenvalue λ_j , the Proportion of Variance

Explained (PVE) by retaining the first q components is:

$$\text{PVE}(q) = \frac{\sum_{j=1}^q \lambda_j}{\sum_{j=1}^p \lambda_j}. \quad (2.5)$$

A common practice is to choose the target dimension q so that the cumulative proportion of variance exceeds a fixed threshold (e.g., 0.95), thereby balancing data compression with information retention.

2.3 Reconstruction and Reconstruction Error

Orthogonal reconstruction

Given Z in (2.4), the rank- q PCA reconstruction in the original feature space is

$$\hat{X}_c = ZU_q^\top = X_c U_q U_q^\top. \quad (2.6)$$

Hence, for the i th centered observation vector $\mathbf{x}_{c,i} = \mathbf{x}_i - \bar{\mathbf{x}}$,

$$\hat{\mathbf{x}}_{c,i} = U_q U_q^\top \mathbf{x}_{c,i},$$

which is precisely the orthogonal projection of $\mathbf{x}_{c,i}$ onto the principal subspace $\text{span}\{\mathbf{u}_1, \dots, \mathbf{u}_q\}$.

Reconstruction error as distance to the principal subspace

Define the reconstruction residual for the i th observation

$$\mathbf{r}_i = \mathbf{x}_{c,i} - \hat{\mathbf{x}}_{c,i} = (I - U_q U_q^\top) \mathbf{x}_{c,i},$$

where $I \in \mathbb{R}^{p \times p}$ is the identity matrix, and its squared magnitude is

$$\|\mathbf{r}_i\|_2^2 = \|\mathbf{x}_{c,i} - U_q U_q^\top \mathbf{x}_{c,i}\|_2^2. \quad (2.7)$$

Geometrically, $\|\mathbf{r}_i\|_2$ is the Euclidean distance from $\mathbf{x}_{c,i}$ to the q -dimensional principal subspace. This quantity forms the basis of PCA reconstruction-error anomaly scoring used later in this thesis.

2.4 A Step-by-Step Numerical Example of PCA and Reconstruction

To illustrate the full PCA pipeline in a concrete and computationally transparent manner, we consider a small two-dimensional dataset consisting of four observations:

$$X = \begin{bmatrix} 2 & 1 \\ 3 & 2 \\ 4 & 2 \\ 5 & 4 \end{bmatrix}.$$

Each row represents a single observation with two features.

Step 1: Compute the Sample Mean

The sample mean vector is

$$\bar{x} = \frac{1}{4} \begin{bmatrix} 2 + 3 + 4 + 5 \\ 1 + 2 + 2 + 4 \end{bmatrix} = \begin{bmatrix} 3.5 \\ 2.25 \end{bmatrix}.$$

Step 2: Center the Data

We subtract the mean from each observation:

$$X_c = X - \mathbf{1}\bar{x}^\top = \begin{bmatrix} -1.5 & -1.25 \\ -0.5 & -0.25 \\ 0.5 & -0.25 \\ 1.5 & 1.75 \end{bmatrix}.$$

Step 3: Compute the Covariance Matrix

The sample covariance matrix is

$$S = \frac{1}{n-1} X_c^\top X_c = \frac{1}{3} X_c^\top X_c.$$

First compute

$$X_c^\top X_c = \begin{bmatrix} 5 & 4.5 \\ 4.5 & 4.75 \end{bmatrix},$$

so that

$$S = \begin{bmatrix} 1.6667 & 1.5 \\ 1.5 & 1.5833 \end{bmatrix}.$$

Step 4: Compute Eigenvalues and Eigenvectors

We solve

$$\det(S - \lambda I) = 0,$$

which yields eigenvalues approximately

$$\lambda_1 \approx 3.125, \quad \lambda_2 \approx 0.125.$$

The corresponding unit eigenvectors are

$$u_1 \approx \begin{bmatrix} 0.72 \\ 0.69 \end{bmatrix}, \quad u_2 \approx \begin{bmatrix} -0.69 \\ 0.72 \end{bmatrix}.$$

The first principal component u_1 captures the dominant direction of variation.

Step 5: Project the Data onto the First Principal Component

Using u_1 , the one-dimensional PCA representation is the score vector

$$z = X_c u_1.$$

Computing each projection:

$$z_1 \approx -1.94, \quad z_2 \approx -0.53, \quad z_3 \approx 0.19, \quad z_4 \approx 2.28.$$

Thus, each observation is now represented by a single scalar value, namely one entry of the score vector z .

Step 6: Reconstruct the Data from the Reduced Representation

The rank-1 reconstruction is given by

$$\hat{X}_c = z u_1^\top.$$

For example, the first reconstructed centered observation is

$$\hat{x}_{c,1} = z_1 u_1 \approx -1.94 \begin{bmatrix} 0.72 \\ 0.69 \end{bmatrix} = \begin{bmatrix} -1.40 \\ -1.34 \end{bmatrix}.$$

Adding back the mean:

$$\hat{x}_1 = \begin{bmatrix} -1.40 \\ -1.34 \end{bmatrix} + \begin{bmatrix} 3.5 \\ 2.25 \end{bmatrix} = \begin{bmatrix} 2.10 \\ 0.91 \end{bmatrix}.$$

Step 7: Compute Reconstruction Error

The reconstruction error for each observation is defined as

$$e_i = \|x_i - \hat{x}_i\|_2^2.$$

We compute the reconstructed points and corresponding errors for all observations:

i	x_i	$\hat{x}_i$	$(x_i - \hat{x}_i)$	$\ x_i - \hat{x}_i\ _2^2$
1	(2, 1)	(2.10, 0.91)	(-0.10, 0.09)	0.018
2	(3, 2)	(3.12, 1.88)	(-0.12, 0.12)	0.029
3	(4, 2)	(3.64, 2.38)	(0.36, -0.38)	0.274
4	(5, 4)	(5.15, 3.82)	(-0.15, 0.18)	0.054

Interpretation

This example demonstrates the complete PCA pipeline:

- The data are centered using the sample mean.
- The covariance matrix captures relationships between features.
- Eigenvectors define directions of maximal variance.
- Projection onto the first principal component reduces dimensionality.
- Reconstruction approximates the original data using only dominant variation.

- Reconstruction error quantifies the information lost during compression.

Although this example is low-dimensional, the same sequence of operations applies directly to high-dimensional datasets. In particular, reconstruction error provides a natural basis for anomaly detection, as observations that lie far from the low-dimensional subspace that best represents the data, i.e., the subspace capturing the largest variation, are reconstructed poorly and therefore exhibit larger errors.

2.5 Low-Dimensional Geometric Intuition

A two-dimensional illustration

In $\mathbb{R}^2$, PCA identifies a new pair of orthonormal axes (u_1, u_2) obtained by rotating the original coordinate system so that u_1 aligns with the direction of maximum spread. If the data are approximately distributed along a line, then u_1 captures most of the variance, while u_2 captures minimal variance. Reducing from 2D to 1D corresponds to projecting points onto the line spanned by u_1 and discarding the orthogonal component.

A three-dimensional illustration

In $\mathbb{R}^3$, PCA produces three orthonormal directions (u_1, u_2, u_3) ordered by explained variance $(\lambda_1, \lambda_2, \lambda_3)$. Reducing from 3D to 2D amounts to projecting onto the plane spanned by (u_1, u_2) and discarding the smallest- variance direction u_3 . This is particularly useful when data form an elongated “cloud” that is difficult to interpret in the original coordinate system.



Figure 2.1: Illustration of PCA in two dimensions: PCA rotates the coordinate system so that PC1 captures the largest variance while PC2 captures the smallest. Projecting onto PC1 yields a one-dimensional representation with minimal loss when PC2 contributes little variance.

2.6 A High-Dimensional Example: A 17-Dimensional Dataset

PCA is often most valuable when p is large and direct visualization is not possible. To illustrate this, consider a dataset where each observation corresponds to a country and each feature corresponds to a measured quantity, producing points in $\mathbb{R}^{17}$. For example, one may treat the weekly per-person consumption of 17 food categories as a vector in $\mathbb{R}^{17}$. Although we cannot visualize $\mathbb{R}^{17}$ directly, PCA provides low-dimensional views (e.g., $q = 1$ or $q = 2$) that reveal dominant contrasts among countries.



Figure 2.2: Illustration of PCA in three dimensions: PC1, PC2, and PC3 form an orthonormal basis with decreasing explained variance. Dropping PC3 projects data onto a best-fit plane.

Let $x_k \in \mathbb{R}^{17}$ denote the centered consumption vector for country k . The first PC score is $z_{k1} = u_1^\top x_k$, and the second is $z_{k2} = u_2^\top x_k$. Countries separated strongly along PC1 exhibit the greatest differences along the dominant consumption pattern, while PC2 captures the second most important pattern not explained by PC1.

2.7 Synthetic anomaly example

To visualize how PCA organizes behavioral observations and highlights deviations from dominant patterns, we construct a small synthetic experiment consisting of 30 two-dimensional data points. In this setup, 27 points represent typical observations concentrated around a



Figure 2.3: A high-dimensional PCA illustration (17 features): PCA reveals dominant axes of variation among observations (countries). Large separation along PC1 indicates an observation that differs substantially along the primary pattern in the data.

common trend, while three points are intentionally placed away from this pattern to represent anomalous behavior.

Original Behavioral Space

We first examine the dataset in the original feature space. Each point corresponds to a single observation described by two behavioral variables. Figure 2.4 shows the dataset together with the two principal component directions computed from the data. Both components are drawn from the center of the dataset to illustrate the dominant directions of variation identified by PCA.



Figure 2.4: Synthetic two-dimensional dataset with principal component directions drawn from the center of the data.

The first component aligns with the primary trend in the data, while the second component captures variation orthogonal to that trend.

Projection into PCA Space

After computing the principal components, each observation is projected onto the PCA coordinate system. In this representation, every point is described by its coordinates along the first and second components.

Figure 2.5 shows the dataset in this transformed space.



Figure 2.5: Dataset represented in the PCA coordinate system where each observation is described by its PC1 and PC2 coordinates.

This representation separates variation along the dominant direction from orthogonal variation.

Principal Component Coordinates

To further illustrate how individual observations relate to the learned components, we plot the principal component values of all data points. Figures 2.6 and 2.7 show the coordinates of each observation along the first and second principal components respectively.



Figure 2.6: Values of the first principal component for all observations.



Figure 2.7: Values of the second principal component for all observations.

These plots show how each observation contributes to the dominant and secondary directions of variation. Points that deviate from the general pattern of the data become more clearly distinguishable when examined through their component coordinates.

Summary of the Illustration

The experiment proceeds through the following steps:

1. Generate a synthetic two-dimensional dataset with 30 observations, including a small number of anomalies.
2. Compute the principal components from the dataset.
3. Visualize the principal directions in the original feature space.

4. Project the observations into the PCA coordinate system.
5. Examine the component coordinates of each observation through line plots.

This sequence of visualizations illustrates how PCA organizes the structure of the dataset and separates dominant behavioral variation from secondary deviations.

2.8 Summary

This chapter presented PCA in the general p -dimensional setting via the covariance matrix, eigendecomposition, and variance-maximization principle. We then interpreted PCA as an orthogonal projection onto a low-dimensional subspace, yielding an explicit reconstruction and a reconstruction-error quantity. The geometric illustrations in low dimensions and the high-dimensional example motivate the role of PCA as both a visualization tool and a foundation for reconstruction-error anomaly scoring developed in later chapters.

Chapter III

Experimental Setup and Datasets

3.1 The CMU CERT Dataset

[General introduction to the CERT dataset, log types, and the concept of malicious scenarios goes here.]

3.2 Data Preparation for Static PCA Analysis

[This section will detail the Deep Feature Synthesis, tabular flattening, and setup for the PCA-GMM pipeline.]

3.3 Data Preparation for Language-Model Analysis

Serialization

Each user-day is written as plain text: one line of organizational attributes, one line of personality scores, a line counting the day’s sessions, and one line per session describing what the user did. Texts longer than 2,048 tokens are truncated. Chapter IV shows an example.

Model and Training

The language model is Qwen3-8B [14], adapted with QLoRA [13] (low-rank adaptation of rank 16 on top of a 4-bit copy of the model). Only benign user-days are used for training; no malicious day is ever shown to the model.

User-Disjoint Evaluation

All results for the language model use CERT release 4.2 [5]. The evaluation pool contains 60 malicious users and 79 benign users, none of whom appear in training: 40,519 user-days, of which 1,309 are malicious. Each malicious user is ranked against the same 79 benign users, a one-class comparison in which no malicious example is used for training.

Chapter IV

Language-Model Surprisal and Its Decomposition

4.1 Overview and Motivation

Chapter II scored an observation by how poorly a low-dimensional linear model reconstructs it. This chapter scores a user-day by how poorly a large language model, fine-tuned only on benign activity, predicts it. Both scores ask the same question: how badly does a model of normal behavior account for this observation? Two ideas from Chapter II carry over. The fine-tuning changes each weight matrix only by a low-rank correction (Section 4.5), and Chapter V decomposes the model’s internal change with a sparse relative of PCA. Scoring activity by how poorly a model trained on normal activity predicts it follows earlier work on log and insider-threat data [9, 10].

The new contribution of this chapter is to take the score apart. We show that it splits exactly into a part describing *who* the user is and a part describing *what* the user did, and that the first part actively hides malicious users.

4.2 Writing a User-Day as Text

Each *user-day*, one user on one calendar day, is serialized as text. The record below is a real benign day from CERT release 4.2 (user AAF0535, day 7). The session line is a single line in the actual text; it is wrapped here only to fit the page.

```
DAY week=1 project=na role=39 b_unit=0 f_unit=4 dept=15 team=11 itadmin=0
```

```

PSY 0=17 C=21 E=36 A=33 N=31

SESSIONS total=1 kept=1

SES idx=0 pc=0 isworkhour=1 isafterhour=0 isweekend=0 isweekendafterhour=0

duration=489 n_concurrent_sessions=1 start_with=1 end_with=1

ses_start=8.933 ses_end=17.083 n_allact=34 n_logon=1 n_usb=0

usb_mean_usb_dur=0 n_file=0 file_n-disk1=0 file_n_exef=0 n_email=3

email_mean_n_atts=1.333 n_http=29 http_n_jobf=0 http_n_cloudf=0

http_n_leakf=0 http_n_hackf=0

```

This user had one session that day: a working-hours session on their usual computer from 8.933 to 17.083 hours (about 8:56 to 17:05), lasting 489 minutes, with 3 emails and 29 web requests. A day with more sessions has one **SES** line per session.

The four kinds of line fall into two classes:

- **Profile lines** (*P*): the **DAY** line (week number, project, role, business unit, functional unit, department, team, and whether the user is an IT administrator) and the **PSY** line (Big-Five personality scores: openness, conscientiousness, extraversion, agreeableness, neuroticism). They describe *who* the user is. Apart from the week number, they are identical on every day of that user.
- **Behavior lines** (*B*): the **SESSIONS** line (how many sessions the user had, and how many were kept) and the **SES** lines (computer, time of day, duration, and counts of logons, USB, file, email and web activity). They describe *what the user did* that day.

What the class of a token means

The model does not read lines; it reads *tokens* $x_1, \dots, x_n$, short pieces of text such as **DAY**, **dept**, or **=**. Numbers are split into single digits, so **dept=15** is read as the four tokens **dept**, **=**, **1**, **5**.

Every token lies on exactly one line, and a token’s *class* is simply the class of that line:

$$c(x_t) = \begin{cases} P & \text{if } x_t \text{ lies on a DAY or PSY line,} \\ B & \text{if } x_t \text{ lies on a SESSIONS or SES line.} \end{cases}$$

The newline that ends a line counts as part of that line. A token’s class is therefore decided by *where* it sits, not by *what* it is. The digit 1 shows this: in the record above it appears in `week=1`, `team=11` and `0=17`, where it is a profile token, and in `total=1` and `n_logon=1`, where it is a behavior token. Same token, different classes.

Table 4.1: Scored tokens by line for the record above. The record has 213 tokens; the first token, `DAY`, is never scored because nothing comes before it.

line	scored tokens	class
DAY	31	<i>P</i>
PSY	23	<i>P</i>
SESSIONS	9	<i>B</i>
SES	149	<i>B</i>
total	$N = 212$	$N_P = 54, N_B = 158$

In this record $54/212 = 25.5\%$ of the scored tokens are profile tokens. The profile lines have roughly the same length every day, while each additional session adds a `SES` line, so the profile share is smaller on busier days. Averaged over the malicious days of CERT release 4.2, about 22% of scored tokens are profile tokens and 78% are behavior tokens.

Which class a surprisal is counted in

Each surprisal ℓ_t (defined in the next section) is the cost of predicting token x_t , and it is counted in the class of x_t , the token being predicted. When the model predicts a behavior

token, it still reads the profile lines above it as context. The split therefore decides which predictions are *counted*, not what the model is allowed to *see*.

4.3 Next-Token Probability and Surprisal

A language model reads a text one token at a time, and at every position it makes a prediction. Before seeing token x_t , it assigns a probability to *every* item in its vocabulary $\mathcal{V}$, the set of all tokens it knows (151,669 of them for our model). These probabilities are nonnegative and add up to 1, so they form a probability distribution over what might come next. The anomaly score needs only one of them: the probability of the token that actually came next,

$$p_{\theta}(x_t \mid x_1, \dots, x_{t-1}) = p_{\theta}(x_t \mid x_{<t}),$$

where θ stands for all the numbers (parameters) inside the model. This section explains, in four steps, how that probability is produced and how it becomes a surprisal. One running example is used throughout: the model has just read `n_logon=` in a session line and must predict the digit that comes next.

Step 1: Summarize the context as a vector

After reading $x_1, \dots, x_{t-1}$, the model condenses everything it has read into a single vector,

$$\mathbf{h}_{t-1} \in \mathbb{R}^d, \quad d = 4096,$$

called the *context vector*. The subscript $t - 1$ records that it summarizes the tokens up to position $t - 1$; it is used to predict token t . The vector is computed by a stack of 36 layers [11]. This chapter treats that stack as a black box, and Chapter V opens it. What matters here is that $\mathbf{h}_{t-1}$ encodes what the model expects to come next.

Step 2: Score every candidate token

Every vocabulary item v has its own learned vector $\mathbf{w}_v \in \mathbb{R}^d$. The model scores v by the dot product of that vector with the context vector:

$$o_v = \mathbf{w}_v^\top \mathbf{h}_{t-1} = \sum_{i=1}^d w_{v,i} h_{t-1,i} = \|\mathbf{w}_v\| \|\mathbf{h}_{t-1}\| \cos \phi_v, \quad (4.1)$$

where ϕ_v is the angle between the two vectors. The score is large when $\mathbf{w}_v$ is long and points in the same direction as the context, near zero when the two are perpendicular, and negative when they point in opposite directions. Stacking the vectors $\mathbf{w}_v$ as the rows of a matrix $W_{\text{out}} \in \mathbb{R}^{|\mathcal{V}| \times d}$ produces every score at once, as one matrix–vector product:

$$\mathbf{o} = W_{\text{out}} \mathbf{h}_{t-1} \in \mathbb{R}^{|\mathcal{V}|}.$$

In machine learning these raw scores are called *logits*.

For the running example, take toy numbers with $d = 2$ and only three candidate tokens:

$$\mathbf{h}_{t-1} = (1.0, 2.0), \quad \mathbf{w}_1 = (0.4, 0.8), \quad \mathbf{w}_2 = (0.2, 0.4), \quad \mathbf{w}_9 = (0.1, 0.0).$$

The scores are

$$\begin{aligned} o_1 &= 0.4(1.0) + 0.8(2.0) = 2.0, & o_2 &= 0.2(1.0) + 0.4(2.0) = 1.0, \\ o_9 &= 0.1(1.0) + 0.0(2.0) = 0.1. \end{aligned}$$

The vector for 1 points exactly along the context and is the longest, so it scores highest. The vector for 2 points the same way but is half as long, so it scores half as much. The vector for 9 is short and points in a different direction ($\cos \phi_9 = 0.447$), so it scores lowest.

Scores are not yet probabilities: they can be any real number, including a negative one, and they do not add up to 1. Step 3 fixes both problems.

Step 3: Turn scores into probabilities with the softmax

The *softmax* function converts scores into probabilities:

$$p_{\theta}(x_t = v \mid x_{<t}) = \frac{e^{o_v}}{\sum_{v' \in \mathcal{V}} e^{o_{v'}}}. \quad (4.2)$$

It performs two operations, each for a reason.

1. **Exponentiate.** e^o is positive for every real number o , and it increases as o increases. Every token therefore receives a positive weight, and a higher score always means a larger weight.
2. **Normalize.** Dividing each weight by the total of all the weights makes the results add up to exactly 1.

In the running example,

$$(e^{2.0}, e^{1.0}, e^{0.1}) = (7.3891, 2.7183, 1.1052), \quad \text{total} = 11.2125,$$

$$p = \left(\frac{7.3891}{11.2125}, \frac{2.7183}{11.2125}, \frac{1.1052}{11.2125} \right) = (0.6590, 0.2424, 0.0986).$$

The model thinks the next digit is most likely 1, less likely 2, and unlikely to be 9.

Four properties explain how to read the softmax.

- **Score differences become probability ratios.** Dividing (4.2) for two tokens a and b cancels the total:

$$\frac{p(a)}{p(b)} = \frac{e^{o_a}}{e^{o_b}} = e^{o_a - o_b}.$$

Raising a token's score by 1 makes it $e \approx 2.718$ times more likely relative to any other token. In the example, $o_1 - o_2 = 1$, and indeed $0.6590/0.2424 = 2.72$.

- **Only differences between scores matter.** Adding the same constant c to every score changes nothing, because $e^{o_v+c} = e^c e^{o_v}$ and the factor e^c cancels between

each weight and the total. Adding 5 to every score in the example gives weights (1096.6332, 403.4288, 164.0219) with total 1664.0839, and exactly the same probabilities (0.6590, 0.2424, 0.0986).

- **It is a “soft” maximum.** If all the scores are equal, for example (1.0, 1.0, 1.0), the probabilities are equal: $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$. As the gaps between scores grow, probability piles onto the largest. Multiplying the example’s scores by 10 gives (20, 10, 1) and probabilities (0.99995, 4.5×10^{-5} , 5.6×10^{-9}), almost all on the top token. The softmax is a smoothed version of “pick the highest score”: it favors the top score, keeps some probability on the others, and the size of the gaps sets how sharply it chooses.
- **No token is ever impossible.** Every probability is positive, so every surprisal is finite. A probability of exactly zero would make the surprisal infinite (Section 4.4), so a single token never seen in training could otherwise dominate a day’s score.

Step 4: From probability to surprisal

The *surprisal* of the actual token x_t is

$$\ell_t = -\log p_\theta(x_t \mid x_{<t}). \quad (4.3)$$

Section 4.4 explains why $-\log p$ is the right measure of surprise.

Substituting the softmax (4.2) shows what surprisal means in terms of the scores:

$$\ell_t = -\log \frac{e^{o_{x_t}}}{\sum_v e^{o_v}} = \log \sum_v e^{o_v} - \log e^{o_{x_t}} = \text{LSE}(\mathbf{o}) - o_{x_t}, \quad (4.4)$$

where $\text{LSE}(\mathbf{o}) = \log \sum_v e^{o_v}$ is called the *log-sum-exp* of the scores. The log-sum-exp is itself a soft maximum of the scores. Write $o_{\max} = \max_v o_v$. The total $\sum_v e^{o_v}$ contains the term

$e^{o_{\max}}$, and it has $|\mathcal{V}|$ terms, none larger than $e^{o_{\max}}$. Therefore

$$e^{o_{\max}} \leq \sum_v e^{o_v} \leq |\mathcal{V}| e^{o_{\max}} \implies o_{\max} \leq \text{LSE}(\mathbf{o}) \leq o_{\max} + \log |\mathcal{V}|.$$

Equation (4.4) therefore says:

surprisal is how far the actual token's score falls below the soft maximum of all the scores.

A token costs little when the model scored it at or near the top, and costs a lot when the model preferred other tokens by a wide margin.

In the running example, $\text{LSE} = \log 11.2125 = 2.4170$, which lies between the largest score, 2.0, and $2.0 + \log 3 = 3.0986$, as the bound requires. Then:

actual next token	score o	$\ell = \text{LSE} - o$	check: $-\log p$
1	2.0	$2.4170 - 2.0 = 0.4170$	$-\log 0.6590 = 0.4170$
2	1.0	$2.4170 - 1.0 = 1.4170$	$-\log 0.2424 = 1.4170$
9	0.1	$2.4170 - 0.1 = 2.3170$	$-\log 0.0986 = 2.3170$

If the log line reads `n_logon=1`, the day pays 0.4170 nats for this token; if it reads `n_logon=9`, it pays 2.3170. The tokens that the benign-trained model scored low are exactly the ones that push a day's anomaly score up.

4.4 Why the Negative Logarithm Measures Surprise

Surprisal, $-\log p$, is the central quantity of this chapter, so it is worth seeing why this particular function measures how surprising an event is, rather than, say, $1 - p$ or $1/p$. We list what a measure of surprise $S(p)$ for an event of probability p ought to satisfy, and show that two natural requirements leave only the logarithm. This is the measure of information introduced by Shannon [17].

Two requirements

(R1) **Rarer is more surprising.** $S(p)$ decreases as p increases.

(R2) **Surprises of independent events add.** If two independent events A and B both happen, the combined surprise should be the surprise of A plus the surprise of B . Because independence means $P(A \text{ and } B) = P(A) P(B)$, this says

$$S(pq) = S(p) + S(q) \quad \text{for all } p, q \in (0, 1]. \quad (4.5)$$

The requirements force the logarithm

Write each probability as $p = e^{-x}$, that is $x = -\ln p \geq 0$, and define $g(x) = S(e^{-x})$. Then (4.5) becomes

$$g(x + y) = S(e^{-x}e^{-y}) = S(e^{-x}) + S(e^{-y}) = g(x) + g(y),$$

Cauchy's functional equation. We solve it one step at a time.

1. Setting $x = y = 0$ gives $g(0) = 2g(0)$, so $g(0) = 0$, that is $S(1) = 0$.
2. Adding x to itself n times gives $g(nx) = n g(x)$ for every positive integer n .
3. Taking $x = 1/m$ in step 2 with $n = m$ gives $g(1) = m g(1/m)$, so $g(1/m) = g(1)/m$, and then $g(n/m) = n g(1/m) = \frac{n}{m} g(1)$. Writing $c = g(1)$, this means $g(r) = c r$ for every positive rational number r .
4. By (R1), S decreases in p , so g increases in x . For any real x , pick rationals $r < x < r'$; then $c r = g(r) \leq g(x) \leq g(r') = c r'$. Letting r and r' approach x squeezes $g(x) = c x$.

Converting back with $x = -\ln p$,

$$S(p) = c(-\ln p) = -c \ln p, \quad c > 0. \quad (4.6)$$

The constant c only chooses the unit. With $c = 1$ the unit is the *nat*, the natural logarithm used throughout this thesis; with $c = 1/\ln 2$ the formula becomes $S(p) = -\log_2 p$, measured in *bits*.

What the formula says

- $S(1) = 0$: an event that was certain carries no surprise.
- $S(p) \rightarrow \infty$ as $p \rightarrow 0$: an event the model thought impossible would be infinitely surprising. This is why the softmax of Section 4.3 never assigns a probability of exactly zero.
- **Equal ratios, equal steps.** Every tenfold drop in probability adds the same amount, $\ln 10 = 2.3026$ nats. The alternatives do not behave this way:

p	$1 - p$	$1/p$	$-\ln p$
0.1	0.9	10	2.3026
0.01	0.99	100	4.6052
0.001	0.999	1000	6.9078

The measure $1 - p$ barely separates a 1-in-100 event from a 1-in-1000 event (0.99 against 0.999). The measure $1/p$ separates them, but it multiplies instead of adding when independent events combine, violating (R2). Only $-\ln p$ does both.

A worked example: coin flips

A fair coin lands heads with $p = 1/2$, a surprise of $-\log_2(1/2) = 1$ bit. Ten heads in a row have probability $(1/2)^{10} = 1/1024$, so

$$S = -\log_2 \frac{1}{1024} = 10 \text{ bits} = 10 \times (1 \text{ bit}).$$

Ten independent flips carry ten times the surprise of one: requirement (R2) at work. In nats the same event costs $10 \ln 2 = 10 \times 0.6931 = 6.9315$, nearly the 6.9078 of a 1-in-1000 event in the table, as it should be, since $1/1024$ is close to $1/1000$.

Surprisal and the negative log-likelihood

In statistics, the probability that a model assigns to the data actually observed is called the *likelihood*; its logarithm is the *log-likelihood*, which is never positive because probabilities are at most 1; and its negative is the *negative log-likelihood*. For a single observed event, the negative log-likelihood is exactly its surprisal. Fitting a model by maximum likelihood (Section 4.5) is therefore the same as making the observed data as unsurprising as possible.

The surprisal of a whole day is its negative log-likelihood

The model also assigns a probability to the whole text of a day. The *chain rule of probability* says that the probability that two events both happen is the probability of the first times the probability of the second given the first: $P(A \text{ and } B) = P(A) P(B | A)$. Applied token by token, the probability of the day's text is the product of each token's probability given the tokens before it:

$$p_{\theta}(x_2, \dots, x_n | x_1) = p_{\theta}(x_2 | x_1) p_{\theta}(x_3 | x_1, x_2) \cdots p_{\theta}(x_n | x_{<n}) = \prod_{t=2}^n p_{\theta}(x_t | x_{<t}).$$

Taking $-\log$ turns the product into a sum:

$$-\log p_{\theta}(x_2, \dots, x_n | x_1) = \sum_{t=2}^n -\log p_{\theta}(x_t | x_{<t}) = \sum_{t=2}^n \ell_t. \quad (4.7)$$

The sum of a day's token surprisals is therefore exactly the surprisal of the day's entire text: its negative log-likelihood under the model. This is the additivity of requirement (R2) above, with one refinement. Tokens are not independent, so each token's surprisal is measured *given*

the tokens before it, and it is these conditional surprisals that add. For example, if a three-token continuation has token probabilities $\frac{1}{2}$, $\frac{1}{2}$ and $\frac{1}{4}$ (each given what came before), the tokens cost 1, 1 and 2 bits, a total of 4 bits; and the whole continuation has probability $\frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{4} = \frac{1}{16}$, whose surprisal is $-\log_2 \frac{1}{16} = 4$ bits. In nats both equal $4 \ln 2 = 2.7726$.

Days differ in length (between about 200 and 530 tokens in CERT release 4.2), and a longer text accumulates more total surprisal simply by being longer. The anomaly score of Section 4.6 therefore divides the total by the number of scored tokens, giving the surprisal per token.

4.5 Learning From Benign Days Only

Fine-tuning chooses θ to minimize the total surprisal of the benign training text,

$$\theta^* = \arg \min_{\theta} \sum_{D \in \mathcal{B}} \sum_t -\log p_{\theta}(x_t^{(D)} | x_{<t}^{(D)}), \quad (4.8)$$

where $\mathcal{B}$ is the set of benign training days and $x^{(D)}$ is the text of day D . Minimizing the total surprisal of the observed data is maximum-likelihood estimation (Section 4.4).

What minimizing surprisal learns

Consider the simplest possible model: one probability p_j for each of K tokens, the same whatever came before. If token j appears N_j times in the benign training text, with $N = \sum_j N_j$, the total surprisal of that text is

$$J(p) = \sum_{j=1}^K N_j (-\log p_j),$$

to be minimized subject to $\sum_j p_j = 1$. With a Lagrange multiplier β ,

$$\frac{\partial}{\partial p_j} \left[\sum_k N_k (-\log p_k) + \beta \left(\sum_k p_k - 1 \right) \right] = -\frac{N_j}{p_j} + \beta = 0, \quad \text{so} \quad p_j = \frac{N_j}{\beta},$$

and the constraint gives $\sum_j N_j / \beta = 1$, so $\beta = N$ and

$$\hat{p}_j = \frac{N_j}{N}. \tag{4.9}$$

Because $-\log$ is convex, this stationary point is the minimum. The probabilities that make the benign data least surprising are simply the benign frequencies.

The same argument works for a model that looks at a fixed piece of context: count only the positions that follow that context. For example, suppose that in the benign training text the digit after `n_logon=` is 1 at 80 positions, 2 at 15 and 9 at 5 (illustrative counts). Then $\hat{p} = (0.80, 0.15, 0.05)$, and the surprisals are

$$-\log 0.80 = 0.2231, \quad -\log 0.15 = 1.8971, \quad -\log 0.05 = 2.9957.$$

A value that is common among benign users costs little, and a value that is rare among them costs a lot. That is exactly why surprisal under a benign-trained model works as an anomaly score. A digit that never followed `n_logon=` in training would get $\hat{p} = 0$ and infinite surprisal; the softmax of Section 4.3 never assigns zero, so this cannot happen to the language model. Fine-tuning performs the same minimization, (4.8), for a model whose probabilities depend on the entire preceding text. No counting formula exists for that model, so the minimum is found numerically, by gradient descent.

Low-rank adaptation

Rather than changing every weight, fine-tuning adds a low-rank correction to the model's weight matrices and keeps the original weights frozen [12]. For a square matrix $W \in \mathbb{R}^{d \times d}$,

such as one of the 4096×4096 attention projections,

$$W' = W + GH, \quad G \in \mathbb{R}^{d \times r}, \quad H \in \mathbb{R}^{r \times d}, \quad r \ll d.$$

With $d = 4096$ and our rank $r = 16$:

$$\underbrace{4096 \times 4096}_{\text{full matrix}} = 16,777,216, \quad \underbrace{4096 \times 16 + 16 \times 4096}_{\text{correction } GH} = 131,072,$$

so the correction trains $131,072/16,777,216 = 0.78\%$ as many numbers. Whatever the input $\mathbf{x}$, the change in the matrix's output, $GH\mathbf{x} = G(H\mathbf{x})$, is a combination of the $r = 16$ columns of G . Just as the rank- q reconstruction of Chapter II keeps only q directions, fine-tuning can change each adapted matrix's output along at most 16 directions. The parallel is structural only: low-rank adaptation limits how the weights may change during training, whereas PCA, and the sparse dictionary of Chapter V, describe data.

4.6 The Anomaly Score of a Day and of a User

Let $\mathcal{T}$ be the set of scored positions of a day (every token except the first, which has nothing before it) and $N = |\mathcal{T}|$. The anomaly score of the day is its average token surprisal,

$$\bar{\ell} = \frac{1}{N} \sum_{t \in \mathcal{T}} \ell_t. \quad (4.10)$$

A high $\bar{\ell}$ means that a model trained only on benign days found this day hard to predict. A user's score is the score of their most unusual day,

$$\bar{\ell}(u) = \max_{D \in \mathcal{D}_u} \bar{\ell}(D),$$

where $\mathcal{D}_u$ is the set of that user's days.

4.7 Splitting the Score by Token Class

Divide the scored positions into profile and behavior positions, $\mathcal{T} = \mathcal{T}_P \cup \mathcal{T}_B$, with $N = N_P + N_B$, and write the class totals and class means as

$$L_P = \sum_{t \in \mathcal{T}_P} \ell_t, \quad L_B = \sum_{t \in \mathcal{T}_B} \ell_t, \quad \bar{\ell}_P = \frac{L_P}{N_P}, \quad \bar{\ell}_B = \frac{L_B}{N_B}.$$

The exact decomposition

Starting from (4.10),

$$\begin{aligned} \bar{\ell} &= \frac{1}{N} \sum_{t \in \mathcal{T}} \ell_t = \frac{1}{N} \left(\sum_{t \in \mathcal{T}_P} \ell_t + \sum_{t \in \mathcal{T}_B} \ell_t \right) = \frac{1}{N} (L_P + L_B) \\ &= \frac{1}{N} (N_P \bar{\ell}_P + N_B \bar{\ell}_B) = \frac{N_P}{N} \bar{\ell}_P + \frac{N_B}{N} \bar{\ell}_B. \end{aligned} \tag{4.11}$$

The day's score is a weighted average of the profile mean and the behavior mean, weighted by their shares of the tokens. Nothing is approximated.

The behavior-only score

The behavior mean can be computed from the day's total and its profile part:

$$\bar{\ell}_B = \frac{L_B}{N_B} = \frac{L - L_P}{N - N_P}, \quad L = L_P + L_B. \tag{4.12}$$

Scoring days by $\bar{\ell}_B$ instead of $\bar{\ell}$ uses the same model, with no retraining; it simply stops counting profile tokens.

Why subtracting the profile mean does not work

A tempting shortcut is $\bar{\ell} - \bar{\ell}_P$. Using (4.11) and $1 - N_P/N = N_B/N$,

$$\begin{aligned}\bar{\ell} - \bar{\ell}_P &= \frac{N_P}{N}\bar{\ell}_P + \frac{N_B}{N}\bar{\ell}_B - \bar{\ell}_P = \frac{N_B}{N}\bar{\ell}_B - \left(1 - \frac{N_P}{N}\right)\bar{\ell}_P \\ &= \frac{N_B}{N}\bar{\ell}_B - \frac{N_B}{N}\bar{\ell}_P = \frac{N_B}{N}(\bar{\ell}_B - \bar{\ell}_P).\end{aligned}\tag{4.13}$$

This measures how much harder the behavior was to predict than the profile, not how hard the behavior was to predict, and it can reverse the order of two days (Step 6 below). Removing only the profile’s weighted share gives

$$\bar{\ell} - \frac{N_P}{N}\bar{\ell}_P = \frac{N_B}{N}\bar{\ell}_B,$$

which is proportional to $\bar{\ell}_B$; because N_B/N varies from day to day, we compute $\bar{\ell}_B$ directly with (4.12).

4.8 A Step-by-Step Numerical Example

Two user-days, each with $N = 5$ scored tokens: one profile token and four behavior tokens, so $N_P/N = 0.2$ and $N_B/N = 0.8$ (close to the real 22%/78% split). Day A belongs to a malicious user with a common profile and unusual behavior. Day B belongs to a benign user with a rare profile and routine behavior.

Step 1: Token Probabilities and Surprisals

token	class	Day A (malicious)		Day B (benign)	
		p	$\ell_t = -\log p$	p	$\ell_t = -\log p$
1	P	0.90	0.1054	0.05	2.9957
2	B	0.50	0.6931	0.80	0.2231
3	B	0.40	0.9163	0.60	0.5108
4	B	0.20	1.6094	0.50	0.6931
5	B	0.60	0.5108	0.80	0.2231

Step 2: Class Totals

Day A: $L_P = 0.1054$, $L_B = 0.6931 + 0.9163 + 1.6094 + 0.5108 = 3.7296$, $L = 3.8350$.

Day B: $L_P = 2.9957$, $L_B = 0.2231 + 0.5108 + 0.6931 + 0.2231 = 1.6501$, $L = 4.6458$.

Step 3: Full Scores

$$\bar{\ell}(A) = \frac{3.8350}{5} = 0.7670, \quad \bar{\ell}(B) = \frac{4.6458}{5} = 0.9292.$$

The full score ranks the benign Day B *above* the malicious Day A.

Step 4: Class Means and the Decomposition Check

$$\begin{aligned} \text{Day A: } \bar{\ell}_P &= 0.1054, \quad \bar{\ell}_B = \frac{3.7296}{4} = 0.9324; \\ \text{Day B: } \bar{\ell}_P &= 2.9957, \quad \bar{\ell}_B = \frac{1.6501}{4} = 0.4125. \end{aligned}$$

Checking (4.11):

$$\bar{\ell}(A) = 0.2(0.1054) + 0.8(0.9324) = 0.02108 + 0.74592 = 0.76700,$$

$$\bar{\ell}(B) = 0.2(2.9957) + 0.8(0.4125) = 0.59914 + 0.33002 = 0.92916.$$

The behavior-only score ranks Day A (0.9324) above Day B (0.4125): the correct order.

Step 5: Where the Full Score Went Wrong

The full scores differ by $0.92916 - 0.76700 = 0.16216$. Splitting that gap by class,

$$\underbrace{(0.59914 - 0.02108)}_{\text{profile part}} + \underbrace{(0.33002 - 0.74592)}_{\text{behavior part}} = 0.57806 - 0.41590 = 0.16216.$$

The behavior part correctly favors Day A by 0.41590, but the profile part favors Day B by 0.57806 and wins. A rare but harmless profile has outweighed genuinely unusual behavior.

Step 6: The Naive Subtraction Reverses a Ranking

Take two further days with the same token counts: Day C with $\bar{\ell}_P = 3.0$, $\bar{\ell}_B = 1.2$, and Day D with $\bar{\ell}_P = 0.2$, $\bar{\ell}_B = 0.8$. The behavior-only score ranks C above D ($1.2 > 0.8$). Then

$$\bar{\ell}(C) = 0.2(3.0) + 0.8(1.2) = 1.56, \quad \bar{\ell}(D) = 0.2(0.2) + 0.8(0.8) = 0.68,$$

$$\text{naive: } \bar{\ell}(C) - \bar{\ell}_P(C) = 1.56 - 3.0 = -1.44, \quad \bar{\ell}(D) - \bar{\ell}_P(D) = 0.68 - 0.2 = 0.48,$$

so the naive subtraction ranks D above C, the wrong order, in agreement with (4.13): $0.8(1.2 - 3.0) = -1.44$ and $0.8(0.8 - 0.2) = 0.48$. Removing only the weighted share restores the order: $1.56 - 0.2(3.0) = 0.96 = 0.8(1.2)$ and $0.68 - 0.2(0.2) = 0.64 = 0.8(0.8)$.

4.9 Measuring Detection: The User-Level AUC

With one malicious user m compared against n_- benign users $u_1, \dots, u_{n_-}$, the area under the ROC curve for m is the fraction of benign users that m outranks,

$$\text{AUC}_m = \frac{1}{n_-} \sum_{j=1}^{n_-} \mathbb{I}[\bar{\ell}(m) > \bar{\ell}(u_j)], \quad (4.14)$$

where $\mathbb{I}[\cdot]$ is 1 when the condition holds and 0 otherwise (a tie counts as $\frac{1}{2}$). Averaging over M malicious users gives $\text{AUC} = \frac{1}{M} \sum_m \text{AUC}_m$. Because every malicious user is compared with the same benign users, this average equals the pooled AUC over all $M \times n_-$ malicious–benign pairs:

$$\frac{1}{M} \sum_m \frac{1}{n_-} \sum_j \mathbb{I}[\cdot] = \frac{1}{M n_-} \sum_m \sum_j \mathbb{I}[\cdot].$$

A value of 0.5 is chance, 1 is perfect, and a value below 0.5 means malicious users are systematically ranked *below* benign ones.

For example, suppose four benign users have full scores (0.93, 0.70, 0.85, 0.60) and a malicious user has full score 0.77. The malicious user outranks 0.70 and 0.60, so $\text{AUC}_m = 2/4 = 0.50$. If the behavior-only scores are (0.41, 0.55, 0.70, 0.48) for the benign users and 0.93 for the malicious user, it outranks all four, and $\text{AUC}_m = 4/4 = 1.00$.

4.10 Core Finding

Table 4.2 scores the same 60 malicious and 79 benign users of CERT release 4.2 with four versions of the score, all computed from one set of model outputs.

Table 4.2: User-level AUC on CERT release 4.2 (60 malicious users, 79 benign users, none seen in training) for scores built from different token classes. Intervals are 95% bootstrap intervals, resampling malicious users.

score	tokens counted	user AUC	95% interval
full score $\bar{\ell}$	all	0.65	[0.58, 0.72]
behavior only $\bar{\ell}_B$	behavior	0.86	[0.83, 0.89]
profile only $\bar{\ell}_P$	profile	0.47	[0.40, 0.54]
organization line only	DAY	0.22	[0.15, 0.29]

Scoring behavior tokens alone raises the user-level AUC by +0.21, with a 95% interval of [+0.16, +0.27]. Compared one malicious user at a time, 51 of the 60 malicious users are ranked higher among the benign users, 3 are unchanged, and 6 are ranked lower.

Three observations explain the result.

- **The profile tokens are not neutral noise.** On their own they rank malicious users below benign ones (0.47), and the organization line alone does so strongly (0.22). In this benchmark, malicious users tend to have common profiles, so their profile tokens are cheap and pull their full scores down: exactly Day A of the worked example.
- **Surprisal measures typicality, not suspicion.** Every evaluated user is new to the model, so a profile’s surprisal reflects how common that kind of employee was among benign training users. A rare but innocent job title is surprising; a malicious user with an ordinary job title is not.
- **The fix requires no new model.** The decomposition (4.11) is exact, so the behavior-only score is obtained from the same model outputs by not counting profile tokens. It removes the profile’s direct contribution to the score, not its influence: the profile is still in the context, so every behavior-token prediction is still conditioned on it.

The same check applies to any detector whose input mixes who a user is with what they did.

The user-level AUC measures how well users are ranked. It does not show that the score pinpoints the specific malicious day, and it should not be read that way.

Chapter V

Interpretable Dimensionality Reduction of the Fine-Tuned Model

5.1 Overview and Motivation

Chapter IV examined the score from the outside: which tokens its surprisal comes from. *Mechanistic interpretability* asks what the model computes internally [15, 16]. This chapter answers that question with a form of dimensionality reduction designed for interpretation. PCA (Chapter II) reduces data to a few dense directions, each of which mixes many original features. A *sparse autoencoder* instead describes each observation with a few directions drawn from a much larger dictionary; because only a few are active at once, each direction, or *feature*, tends to mean one recognizable thing. We apply it to the change that benign fine-tuning made to the model’s internal representations, and test two things about the features that best separate malicious from benign days: where they activate, and whether editing them changes the anomaly score.

5.2 Hidden States and the Adaptation Difference

The model processes text through a stack of 36 layers. After each layer, each token t is represented by a *hidden state* $\mathbf{h}_t \in \mathbb{R}^d$ with $d = 4096$. Running the same text through the model before fine-tuning (the *base* model) and after it (the *adapted* model) gives two hidden

states for every token. Their difference,

$$\boldsymbol{\delta}_t = \mathbf{h}_t^{\text{ada}} - \mathbf{h}_t^{\text{base}} \in \mathbb{R}^d, \quad (5.1)$$

is exactly what benign fine-tuning added to the representation of token t . We study the hidden states after layer 26. For instance, if $\mathbf{h}_t^{\text{base}} = (0.5, 1.0, -0.2)$ and $\mathbf{h}_t^{\text{ada}} = (0.9, 1.0, 0.4)$ in a three-dimensional illustration, then $\boldsymbol{\delta}_t = (0.4, 0.0, 0.6)$: fine-tuning changed the first and third coordinates and left the second alone.

5.3 A Sparse Dictionary for the Difference

Recall the PCA reconstruction of Chapter II: an observation is encoded by $\mathbf{z} = U_q^\top \mathbf{x}$ using $q < p$ orthonormal directions, decoded by $\hat{\mathbf{x}} = U_q \mathbf{z}$, and the directions minimize the reconstruction error (2.7). A *sparse autoencoder* (SAE) keeps the reconstruction objective but changes two things: it uses *more* directions than dimensions ($m > d$), and it lets only a few of them be active for any one token [18].

Encode. Compute a score for each of the m dictionary directions and keep only the k largest:

$$\mathbf{a}_t = W_{\text{enc}}(\boldsymbol{\delta}_t - \mathbf{b}) \in \mathbb{R}^m, \quad \mathbf{z}_t = \text{TopK}_k(\mathbf{a}_t), \quad (5.2)$$

where TopK_k keeps the k largest entries and sets the rest to zero.

Decode. Rebuild the difference from the active directions:

$$\hat{\boldsymbol{\delta}}_t = W_{\text{dec}} \mathbf{z}_t + \mathbf{b} = \sum_{f: z_{f,t} \neq 0} z_{f,t} \mathbf{w}_f + \mathbf{b}, \quad (5.3)$$

where $\mathbf{w}_f$ is column f of W_{dec} , the direction of *feature* f .

Train. Choose W_{enc} , W_{dec} and $\mathbf{b}$ to minimize $\sum_t \|\boldsymbol{\delta}_t - \widehat{\boldsymbol{\delta}}_t\|_2^2$, the same squared reconstruction error as PCA. No labels are used. (Each coordinate of $\boldsymbol{\delta}$ is standardized before encoding and restored after decoding; we suppress this in the notation.)

Table 5.1: PCA and the sparse autoencoder side by side.

	PCA (Chapter II)	sparse autoencoder
number of directions	$q < p$	$m > d$
directions	orthonormal	not orthogonal
active per observation	all q	only k
objective	$\min \ \mathbf{x} - \widehat{\mathbf{x}}\ ^2$	$\min \ \boldsymbol{\delta} - \widehat{\boldsymbol{\delta}}\ ^2$
our setting	—	$d = 4096, m = 8192, k = 4$

With $m = 8,192$ directions but only $k = 4$ active per token, each token’s $\boldsymbol{\delta}_t$ is explained by four features. Because so few are active at once, each feature tends to specialize in one recognizable pattern. Our dictionary is trained on 2,000,000 token-level differences.

5.4 A Step-by-Step Numerical Example of Sparse Coding

Take $d = 2$, a dictionary of $m = 3$ unit directions, $k = 1$ active feature, $\mathbf{b} = \mathbf{0}$, and, for illustration, $W_{\text{enc}} = W_{\text{dec}}^\top$:

$$\mathbf{w}_1 = (1, 0), \quad \mathbf{w}_2 = (0, 1), \quad \mathbf{w}_3 = (0.6, 0.8).$$

Step 1: Encode

For $\boldsymbol{\delta} = (1.2, 1.6)$,

$$\mathbf{a} = (\mathbf{w}_1^\top \boldsymbol{\delta}, \mathbf{w}_2^\top \boldsymbol{\delta}, \mathbf{w}_3^\top \boldsymbol{\delta}) = (1.2, 1.6, 0.6(1.2) + 0.8(1.6)) = (1.2, 1.6, 2.0).$$

Step 2: Keep the Top $k = 1$

The largest score belongs to feature 3, so $\mathbf{z} = (0, 0, 2.0)$.

Step 3: Decode and Measure the Error

$$\hat{\boldsymbol{\delta}} = 2.0 \mathbf{w}_3 = (1.2, 1.6), \quad \|\boldsymbol{\delta} - \hat{\boldsymbol{\delta}}\|^2 = 0.$$

Step 4: Compare With the Two Axes Alone

If only $\mathbf{w}_1$ and $\mathbf{w}_2$ were available, with one active at a time, the best choice is $\mathbf{w}_2$: $\hat{\boldsymbol{\delta}} = (0, 1.6)$ with error $1.2^2 = 1.44$ (using $\mathbf{w}_1$ gives $1.6^2 = 2.56$). The larger dictionary represents $\boldsymbol{\delta}$ exactly with one active feature because it contains a direction aligned with it.

Step 5: A Different Token Uses a Different Feature

For $\boldsymbol{\delta}' = (1.0, 0.1)$, $\mathbf{a}' = (1.0, 0.1, 0.68)$, so feature 1 is kept: $\hat{\boldsymbol{\delta}}' = (1.0, 0)$ with error $0.1^2 = 0.01$. Each token is described by the few features that fit it best: this is *sparse coding*.

5.5 Choosing the Features to Test

For each feature f , its *gap* compares how strongly it fires on malicious and benign days,

$$g_f = \bar{z}_f^{(+)} - \bar{z}_f^{(-)}, \tag{5.4}$$

where $\bar{z}_f^{(+)}$ and $\bar{z}_f^{(-)}$ are the average activations of feature f over the tokens of malicious days and of benign days. The *selected set* $\mathcal{F}$ is the five features with the largest gaps. The *control set* $\mathcal{C}$ is the five features with the gaps closest to zero among those active on at least 0.2% of tokens. The threshold keeps rarely firing features out of the control set, but it does not match the two sets' activity: in the held-out test below, the control features were active on 0.36% of tokens on average and the selected features on 0.18%. The control answers the question: does editing *any* active features change the score, or only the selected ones?

5.6 Where Features Fire: Token-Class Attribution

For a feature f and a token class c , the share of the feature's total activation that lands on class- c tokens is

$$\rho_f(c) = \frac{\sum_{t \in \mathcal{T}_c} z_{f,t}}{\sum_{t \in \mathcal{T}} z_{f,t}}, \quad (5.5)$$

and its *enrichment* compares that share with the class's share of the tokens,

$$E_f(c) = \frac{\rho_f(c)}{N_c/N}.$$

An enrichment of 1 means the feature fires on class c exactly as often as chance would suggest; larger values mean it prefers class c .

For example, if a feature's activations over five tokens of classes (P, P, B, B, B) are $(0, 0, 1.5, 2.0, 0.5)$, the total is 4.0, so $\rho_f(B) = 4.0/4.0 = 1.00$ and $\rho_f(P) = 0$. Behavior tokens are $3/5 = 0.60$ of the tokens, so $E_f(B) = 1.00/0.60 = 1.67$. The largest possible enrichment for a class is $1/(N_c/N)$, reached when all of a feature's activation lands on that class.

5.7 Causal Test: Editing Features and Re-Scoring

Attribution shows where a feature fires, not whether it matters. The causal test edits the features and measures the change in the anomaly score. Take a malicious *receiver* day. A *donor* day comes from another user in the same department, and it is one of two kinds: a benign day of a colleague, or a malicious day of another malicious user. Comparing the two kinds of donor asks whether the benign colleague’s values in particular make the receiver look normal, rather than any edit of the same features.

Step 1: Encode the receiver

Compute $\mathbf{z}_t$ for each receiver token with (5.2).

Step 2: Move the selected features toward the donor

On tokens where the selected features are active, replace each selected feature’s activation by

$$z'_{f,t} = (1 - \gamma) z_{f,t} + \gamma \bar{z}_f^{\text{donor}}, \quad f \in \mathcal{F},$$

with patch strength $\gamma \in \{0.25, 0.5, 0.75, 1\}$; all other features are unchanged. Here $\bar{z}_f^{\text{donor}}$ is the average activation of feature f over the donor day’s tokens on which at least one selected feature is active. For example, if $z_{f,t} = 2.0$ and $\bar{z}_f^{\text{donor}} = 0.4$, then $\gamma = 1$ gives $z'_{f,t} = 0.4$, and $\gamma = 0.5$ gives $z'_{f,t} = 0.5(2.0) + 0.5(0.4) = 1.2$.

Step 3: Decode and replace the hidden state

Decode the edited code, $\widehat{\boldsymbol{\delta}}'_t = W_{\text{dec}} \mathbf{z}'_t + \mathbf{b}$, and replace the adapted hidden state at layer 26 by

$$\mathbf{h}'_t = \mathbf{h}_t^{\text{ada}} + (\widehat{\boldsymbol{\delta}}'_t - \boldsymbol{\delta}_t).$$

Substituting $\mathbf{h}_t^{\text{ada}} = \mathbf{h}_t^{\text{base}} + \boldsymbol{\delta}_t$ from (5.1),

$$\mathbf{h}'_t = \mathbf{h}_t^{\text{base}} + \boldsymbol{\delta}_t + \widehat{\boldsymbol{\delta}}'_t - \boldsymbol{\delta}_t = \mathbf{h}_t^{\text{base}} + \widehat{\boldsymbol{\delta}}'_t.$$

If any token of the day has an active selected feature, this replacement is made at *every* token of the day: tokens without an active selected feature receive their unedited reconstruction $W_{\text{dec}}\mathbf{z}_t + \mathbf{b}$, and only the active tokens carry the edit. A day with no active selected feature is left unchanged. The reconstruction is not exact, so it can move the score even without an edit; Step 6 is built so that this movement cancels.

Step 4: Re-score

Run the remaining layers from $\mathbf{h}'$ and recompute the day's score $\bar{\ell}'$. The change in the score is $\delta = \bar{\ell}' - \bar{\ell}$; a negative change means the edit made the day look less anomalous.

Step 5: Keep the best candidate

For each receiver day, up to 16 candidate donor days of each kind are tried, each at all four patch strengths. For each kind of donor, the most negative change is kept, giving $\delta_{\mathcal{F},\text{ben}}$ and $\delta_{\mathcal{F},\text{mal}}$. Repeating Steps 2–5 with the control set gives $\delta_{\mathcal{C},\text{ben}}$ and $\delta_{\mathcal{C},\text{mal}}$.

Step 6: Compare donors, then compare feature sets

For each feature set, the *donor advantage* is how much more the score falls with a benign donor than with a malicious one:

$$a_{\mathcal{F}} = \delta_{\mathcal{F},\text{mal}} - \delta_{\mathcal{F},\text{ben}}, \quad a_{\mathcal{C}} = \delta_{\mathcal{C},\text{mal}} - \delta_{\mathcal{C},\text{ben}}.$$

The effect of the selected features is the average, over receiver days that have all four values, of

$$\tau = a_{\mathcal{F}} - a_{\mathcal{C}} = (\delta_{\mathcal{F},\text{mal}} - \delta_{\mathcal{F},\text{ben}}) - (\delta_{\mathcal{C},\text{mal}} - \delta_{\mathcal{C},\text{ben}}). \quad (5.6)$$

Two things cancel. The unedited score $\bar{\ell}$ appears in all four changes. And within one feature set, the benign-donor and malicious-donor edits reconstruct the same tokens of the same day, so the reconstruction’s own effect is the same in both and cancels in $a_{\mathcal{F}}$ and in $a_{\mathcal{C}}$. What remains compares the edits alone.

For example, if $\delta_{\mathcal{F},\text{ben}} = -0.040$, $\delta_{\mathcal{F},\text{mal}} = -0.028$, $\delta_{\mathcal{C},\text{ben}} = -0.012$ and $\delta_{\mathcal{C},\text{mal}} = -0.010$, then

$$\begin{aligned} a_{\mathcal{F}} &= -0.028 - (-0.040) = 0.012, & a_{\mathcal{C}} &= -0.010 - (-0.012) = 0.002, \\ \tau &= 0.012 - 0.002 = 0.010. \end{aligned}$$

A positive τ means that moving the selected features toward a benign colleague, rather than toward another malicious user, lowers the score by more than the same comparison does for the control features.

A simpler comparison uses benign donors only: $\delta_{\mathcal{C},\text{ben}} - \delta_{\mathcal{F},\text{ben}}$, which is 0.028 in the example. It is a different quantity. It mixes the effect of moving toward benign values with any effect of editing the selected features at all, whatever the donor, which τ removes. The interval reported below belongs to τ and cannot be attached to this simpler comparison.

5.8 Core Mechanistic Finding

Where the selected features activate

Table 5.2 gives the attribution (5.5) of the five features selected on all 60 malicious users of CERT release 4.2.

Table 5.2: Share of each selected feature’s activation on each token class, over malicious days of CERT release 4.2 (layer 26). Session (SES) lines are 74.9% of these tokens, the organization (DAY) line 12.6%, and the personality (PSY) line 9.0%.

feature	$\rho_f(\text{SES})$	$\rho_f(\text{DAY})$	$\rho_f(\text{PSY})$	enrichment
4596	0.999	0.000	0.001	1.334 on SES
3673	1.000	0.000	0.000	1.335 on SES
3455	1.000	0.000	0.000	1.335 on SES
1268	1.000	0.000	0.000	1.335 on SES
2302	0.001	0.998	0.001	7.907 on DAY

Four of the five features put at least 99.9% of their activation on session tokens. Their enrichment of 1.334 to 1.335 is the maximum possible, since $1/0.7492 = 1.335$: essentially all of their activation lands on the session lines. The strongest activations of one of them (3673) all fall on the `_dur` token that names the session-duration fields. The fifth feature (2302) fires on the organization line instead.

This table answers one question only: *where* each feature activates. It does not say what information a feature represents; a feature that activates on session tokens could still carry information about who the user is, such as the session lengths typical of a department. Nor does it say which predictions change when the feature is edited. Those are separate questions.

Editing them changes the score, on held-out users

For the causal test, the 60 malicious users were split in half. Features were re-selected using only the 30 *discovery* users; the re-selected set shares four of the five features above. The test was then run on the other 30 malicious users, whom feature selection never saw. With

donors from the same department, (5.6) gave

$$\tau = 0.00076 \text{ nats}, \quad 95\% \text{ interval } [0.00035, 0.00121],$$

over the 605 held-out malicious days with all four values, and τ was positive for 21 of 29 held-out users. The interval resamples users. In the same data the donor advantage was $a_{\mathcal{F}} = 0.00063$ for the selected features and $a_{\mathcal{C}} = -0.00012$ for the controls. The benign-only comparison of Step 6 is 0.0028 here; it has no interval of its own. The estimate is positive in all four organizational groupings tested (department, department and role, team, role: 0.00034 to 0.00076 nats); the department grouping is the one whose interval excludes zero.

Reading

Two things are established. The features that most distinguish malicious days activate almost only on session tokens. And on users held out from feature selection, the choice between a benign and a malicious donor matters more when the selected features are edited than when the control features are, by a small margin. Three things are not established: what information the features represent; which token predictions their edit changes, since the edit is measured here on the full score only and an experiment that splits the same edits by token class is in progress; and whether the profile shortcut of Chapter IV arises mainly from averaging surprisal over every token, since the score decomposition shows that the shortcut is removable at scoring time but not where it arises inside the model.

The effect is small in absolute terms (0.00076 nats against day scores near 1), and it is about half the size of the estimate obtained before the held-out split. The claim is about the direction of the effect and where the features fire, not about its size.

5.9 Limitations

- The dictionary’s layer, size and sparsity (layer 26, $m = 8,192$, $k = 4$) were chosen among twelve candidate settings using runs that included all 60 malicious users. The features were re-selected on held-out users, but this setting was not.
- All results come from a single fine-tuned model, and their stability across training seeds is untested. A companion-dataset result once cited here as a seed reversal selected its features from an undefined ranking and has been withdrawn.
- Step 5 keeps the best of many candidates. That favours whichever set’s candidate scores vary more, and there can be fewer malicious than benign donors, so τ can be positive without a donor-specific effect. The planned analysis averages over all candidates instead.
- The controls pass an activity threshold but are not activity-matched to the selected features.
- One benchmark release was used, CERT release 4.2. It is synthetic, and it was evaluated many times during development.
- The user-level AUC ranks users; it does not show that the model identifies the specific malicious day.

REFERENCES

Bibliography

- [1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11), 559–572.
- [2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.
- [3] Jolliffe, I. T. (1986). *Principal Component Analysis*. Springer Series in Statistics. Springer, New York. <https://doi.org/10.1007/978-1-4757-1904-8>
- [4] Software Engineering Institute. (2016). *Insider Threat Test Dataset*. Carnegie Mellon University Software Engineering Institute. <https://doi.org/10.1184/R1/12841247.v1>
- [5] Glasser, J., & Lindauer, B. (2013). Bridging the gap: A pragmatic approach to generating insider threat data. In *2013 IEEE Security and Privacy Workshops* (pp. 98–104). IEEE. <https://doi.org/10.1109/SPW.2013.37>
- [6] Shlens, J. (2014). *A Tutorial on Principal Component Analysis*. arXiv. <https://doi.org/10.48550/arXiv.1404.1100>
- [7] Cappelli, D., Moore, A., & Trzeciak, R. (2012). *The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes (Theft, Sabotage, Fraud)*. Addison-Wesley Professional.

- [8] HPC Wiki, The University of Southern Mississippi. (2024). *Main Page*. Magnolia HPC Wiki. Suggested infrastructure acknowledgment: The authors acknowledge HPC at The University of Southern Mississippi supported by the National Science Foundation under the Major Research Instrumentation (MRI) program via Grant # ACI 1626217. http://magnolia.usm.edu/index.php?title=Main_Page&oldid=1116
- [9] Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*.
- [10] Tuor, A., Kaplan, S., Hutchinson, B., Nichols, N., & Robinson, S. (2017). Deep learning for unsupervised insider threat detection in structured cybersecurity data streams. In *AAAI Workshop on Artificial Intelligence for Cyber Security*.
- [11] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*.
- [12] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.
- [13] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*.
- [14] Qwen Team. (2025). *Qwen3 technical report*. arXiv preprint arXiv:2505.09388.
- [15] Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N. L., et al. (2023). Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*. <https://transformer-circuits.pub/2023/monosemantic-features>

- [16] Cunningham, H., Ewart, A., Riggs, L., Huben, R., & Sharkey, L. (2023). Sparse autoencoders find highly interpretable features in language models. arXiv preprint arXiv:2309.08600.
- [17] Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423.
- [18] Gao, L., Dupré la Tour, T., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., & Wu, J. (2024). Scaling and evaluating sparse autoencoders. arXiv preprint arXiv:2406.04093.